

1.A

```
1.Start.
2.Declare variables and get str1 , str2.
3.Start if (len(str1) ≠ len(str2)) -> print("0")
    And go to 8          END IF
4. else    strLength <-(=) len(str1)
5. start for (i=0 -> i<strLength -> i= i+1 )
    Start If (str1[i]≠str2[i]) -> print("0")
    And go to 8          End IF
6.         else print("1");
7. END FOR
8. END
```

1.B

```
int getLength(char *str)
{
    int i = 0;
    int strLength = 0;
    while (*(str + i) != '\0')
    {
        strLength++;
        i++;
    }
    return strLength;
}

int compareStrings(char *str1, char *str2)
{
    if (getLength(str1) != getLength(str2))
        return 0;
    int strLength = getLength(str1);

    for (int i = 0; i < strLength; i++)
    {
        if (str1[i] != str2[i])
            return 0;
    }

    return 1;
}
```

2.A

```
typedef struct Sandwich
{
    char *name;
    int price;
    int discount;
} Sandwich;
```

2.B

```
Sandwich* searchSandwich(char* name, Sandwich* dishes, int
menu_size) {
    for (int i = 0; i < menu_size; i++)
    {
        if (compareStrings(name , dishes[i].name)) return
&dishes[i];
    }
    return NULL;
}
```

2.C

```
void canAfford(int money, char *name, Sandwich *dishes, int
menu_size)
{
    Sandwich *searchResult = searchSandwich(name, dishes,
menu_size);
    if (searchResult == NULL)
    {
        printf("Sandwich not found\n");
    }

    else
    {
        if (money < searchResult->price)
        {
            printf("Money is not enough\n");
        }
        else
        {
            printf("Sandwich: %s\nPrice: %d\nDiscounted Price: %d",
searchResult->name, searchResult->price, searchResult->price -
searchResult->discount);
        }
    }
}
```

3.A

```
#include <stdio.h>
#include <string.h>
#define MAX 100
typedef struct
{
    int plate;
    char street[50];
    int unit;
} House;
```

3.B

```
typedef struct
{
    House house[MAX];
    int appliedHouses;
} Registry;
```

3.C

```
int registerHouse(Registry *reg, int plate, char street[], int
unit)
{
    if (reg->appliedHouses >= MAX) return -1;

    for (int i = 0; i < reg->appliedHouses; i++)
    {
        if (reg->house[i].plate == plate) return 0;
    }

    reg->house[reg->appliedHouses].plate = plate;
    reg->house[reg->appliedHouses].unit = unit;
    strcpy(reg->house[reg->appliedHouses].street, street);

    reg->appliedHouses ++;
    return 1;
}
```

4.A

```
typedef struct
{
    int id;
    char sender[20];
    char msg[100];
} Message;
```

4.B

```
typedef struct queue
{
    Message messages[CAP];
    int next;
    int size;
} Queue;
```

4.C

```
void initQueue(Queue *q)
{
    q->next = 0;
    q->size = 0;
}
```

4.D

```
int isEmpty(Queue *q)
{
    if (q->size == 0)
        return 1;
    else
        return 0;
}

int isFull(Queue *q)
{
    if (q->size == CAP)
        return 1;
    return 0;
}
```

٤.E

```
int enqueue(Queue *q, Message msg)
{
    if (isFull(q))
        return 0;
    else
    {
        q->messages[q->next] = msg;
        q->next++;
        q->size++;
    }
    return 1;
}
```

٤.F

```
Message *dequeue(Queue *q)
{
    if (isEmpty(q))
        return NULL;

    Message *deleted = (Message*)malloc(sizeof(Message));

    deleted->id = q->messages[0].id;
    strcpy(deleted->msg, q->messages[0].msg);
    strcpy(deleted->sender, q->messages[0].sender);

    for (int i = q->size - 1; i > 0; i--)
    {
        q->messages[i-1] = q->messages[i];
    }
    q->size--;
    q->next--;
    return deleted;
}
```